

MOVIE RECOMMENDATION ENGINE (NO COOL NAME YET)

Proposal (problem statement + what you're building)

Problem Statement

Choosing a movie has become increasingly difficult due to the overwhelming number of options available across streaming platforms. Users often spend more time browsing than actually watching, leading to decision fatigue and, in many cases, abandoning the search altogether. The problem is even more pronounced for groups, where finding a movie that satisfies everyone's preferences can be time-consuming and frustrating.

Proposed Solution

Develop a web application that provides personalized movie recommendations based on user preferences. Users can either request recommendations individually or create a group session where multiple users submit their preferences. The application analyzes these preferences, retrieves relevant movie information from the TMDb API, and recommends movies with a compatibility score indicating how well each recommendation matches the selected preferences.

SOW (scope of work — what's in, what's explicitly out)

In Scope

- If individual, only user profile with saved preferences
- If group, create room with up to 4 participants and each participant's user profiles taken into account
- Individual preference form including:
 - Favourite movies
 - Preferred genres
 - Preferred runtime
 - Mood
 - Keywords/themes
 - Favourite actors/directors (optional)
 - Free-text preferences (e.g. "I don't want anything too sappy" or "I want something starring Ryan Gosling")
- Recommendation engine using TMDb movie data (based on combined preferences for groups)
- Compatibility score for each recommendation
- Save user preferences for future sessions

Stretch Goals

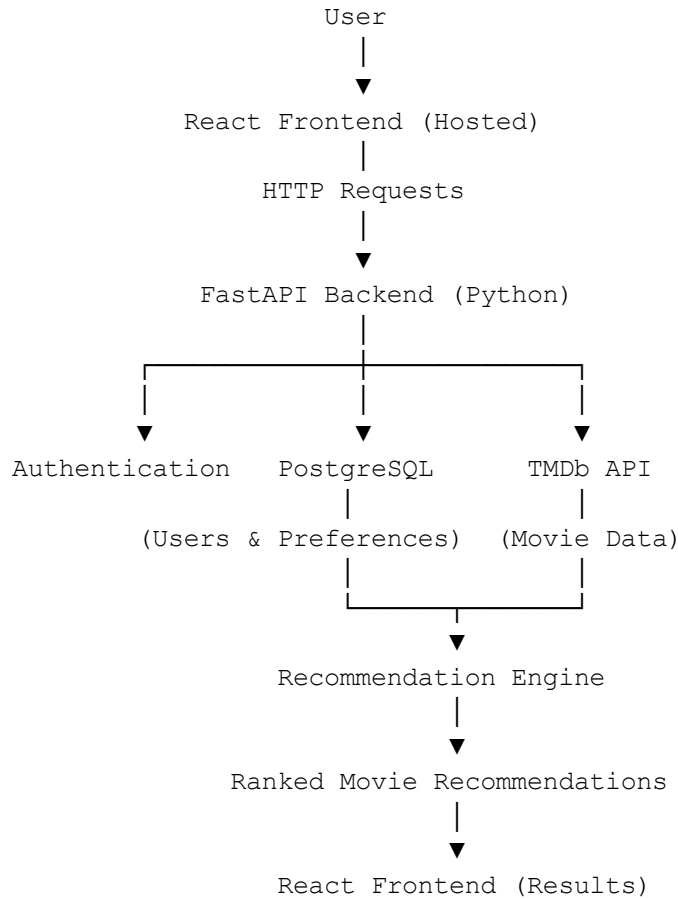
- Display streaming availability ("Where to Watch")
- Like/dislike recommendations to improve future suggestions
- AI-generated explanation for recommendations (e.g. *"Recommended because your group enjoys sci-fi thrillers with strong female leads."*)
- Import data from a Letterboxd account (rather than direct account integration)

Out of Scope

- Letterboxd account login or API integration
- Automatically tracking watched movies
- Personal watchlists

- Movie diary or review functionality
- Social features (friends, followers, comments)

Architecture (a simple diagram is fine)



Tech stack (languages, frameworks, services)

Frontend

- React
- TypeScript
- Tailwind CSS

Backend

- FastAPI (Python)

Database

- PostgreSQL

APIs

- TMDb API

Recommendation Engine

- Python-based recommendation algorithm
- LLM integration (to interpret free-text preferences or explaining recommendations)

Authentication

- JWT authentication

Deployment

- Frontend: Vercel

- Backend: Render (or AWS if time permits)
- Database: PostgreSQL

Anything else relevant to understanding your approach

How the Recommendation Algorithm will Pick Movies:

- Collect user preferences, including favourite movies, genres, preferred runtime, mood, and optional free-text preferences
- Use an LLM to extract meaningful keywords and themes from free-text inputs
- Retrieve candidate movies from the TMDb API based on these preferences
- Expand the candidate pool using TMDb's **Similar Movies** and **Keywords** features
- Calculate a compatibility score for each movie based on factors such as genre, runtime, keywords, similarity to favourite movies, and other matching preferences
- Rank movies by compatibility and return the top 10 recommendations
- Allow users to request additional recommendations if they are not satisfied with the initial results